

Agentic Design Patterns

*A Hands-On Guide to Building Intelligent Systems*¹, [Antonio Gulli](#)

Table of Contents - total 424 pages = 1+2+1+1+4+9+103+61+34+114+74+5+4 11

[Dedication](#), 1 page

[Acknowledgment](#), 2 pages [final, last read done]

[Foreword](#), 1 page [final, last read done]

[A Thought Leader's Perspective: Power and Responsibility](#) [final, last read done]

[Introduction](#), 4 pages [final, last read done]

[What makes an AI system an "agent"?](#), 9 pages [final, last read done]

Part One, (Total: 103 pages)

1. [Chapter 1: Prompt Chaining](#) (code), 12 pages [final, last read done, code ok]
2. [Chapter 2: Routing](#) (code), 13 pages [final, last read done, code ok]
3. [Chapter 3: Parallelization](#) (code), 15 pages [final, last read done, code ok]
4. [Chapter 4: Reflection](#) (code), 13 pages [final, last read done, code ok]
5. [Chapter 5: Tool Use](#) (code), 20 pages [final, last read done, code ok]
6. [Chapter 6: Planning](#) (code), 13 pages [final, last read done, code ok]
7. [Chapter 7: Multi-Agent](#) (code), 17 pages [final, last read done, code ok], **121**

Part Two (Total: 61 pages)

8. [Chapter 8: Memory Management](#) (code), 21 pages [final, last read done, code ok]
9. [Chapter 9: Learning and Adaptation](#) (code), 12 pages [final, last read done, code ok]
10. [Chapter 10: Model Context Protocol \(MCP\)](#) (code), 16 pages [final, last read done, code ok]
11. [Chapter 11: Goal Setting and Monitoring](#) (code), 12 pages [final, last read done, code ok], **182**

Part Three (Total: 34 pages)

12. [Chapter 12: Exception Handling and Recovery](#) (code), 8 pages [final, last read done, code ok]
13. [Chapter 13: Human-in-the-Loop](#) (code), 9 pages [final, last read done, code ok]
14. [Chapter 14: Knowledge Retrieval \(RAG\)](#) (code), 17 pages [final, last read done, code ok], **216**

Part Four (Total: 114 pages)

15. [Chapter 15: Inter-Agent Communication \(A2A\)](#) (code), 15 pages [final, last read done, code ok]
16. [Chapter 16: Resource-Aware Optimization](#) (code), 15 pages [final, last read done, code ok]
17. [Chapter 17: Reasoning Techniques](#) (code), 24 pages [final, last read done, code ok]
18. [Chapter 18: Guardrails/Safety Patterns](#) (code), 19 pages [final, last read done, code ok]
19. [Chapter 19: Evaluation and Monitoring](#) (code), 18 pages [final, last read done, code ok]
20. [Chapter 20: Prioritization](#) (code), 10 pages [final, last read done, code ok]
21. [Chapter 21: Exploration and Discovery](#) (code), 13 pages [final, last read done, code ok], **330**

Appendix (Total: 74 pages)

22. [Appendix A: Advanced Prompting Techniques](#), 28 pages [final, last read done, code ok]
23. [Appendix B - AI Agentic: From GUI to Real world environment](#), 6 pages [final, last read done, code ok]
24. [Appendix C - Quick overview of Agentic Frameworks](#), 8 pages [final, last read done, code ok]
25. [Appendix D - Building an Agent with AgentSpace \(on-line only\)](#), 6 pages [final, last read done, code ok]
26. [Appendix E - AI Agents on the CLI \(online\)](#), 5 pages [final, last read done, code ok]
27. [Appendix F - Under the Hood: An Inside Look at the Agents' Reasoning Engines](#), 14 pages [final, lrd, code ok]
28. [Appendix G - Coding agents](#), 7 pages **406**

[Conclusion](#), 5 pages [final, last read done]

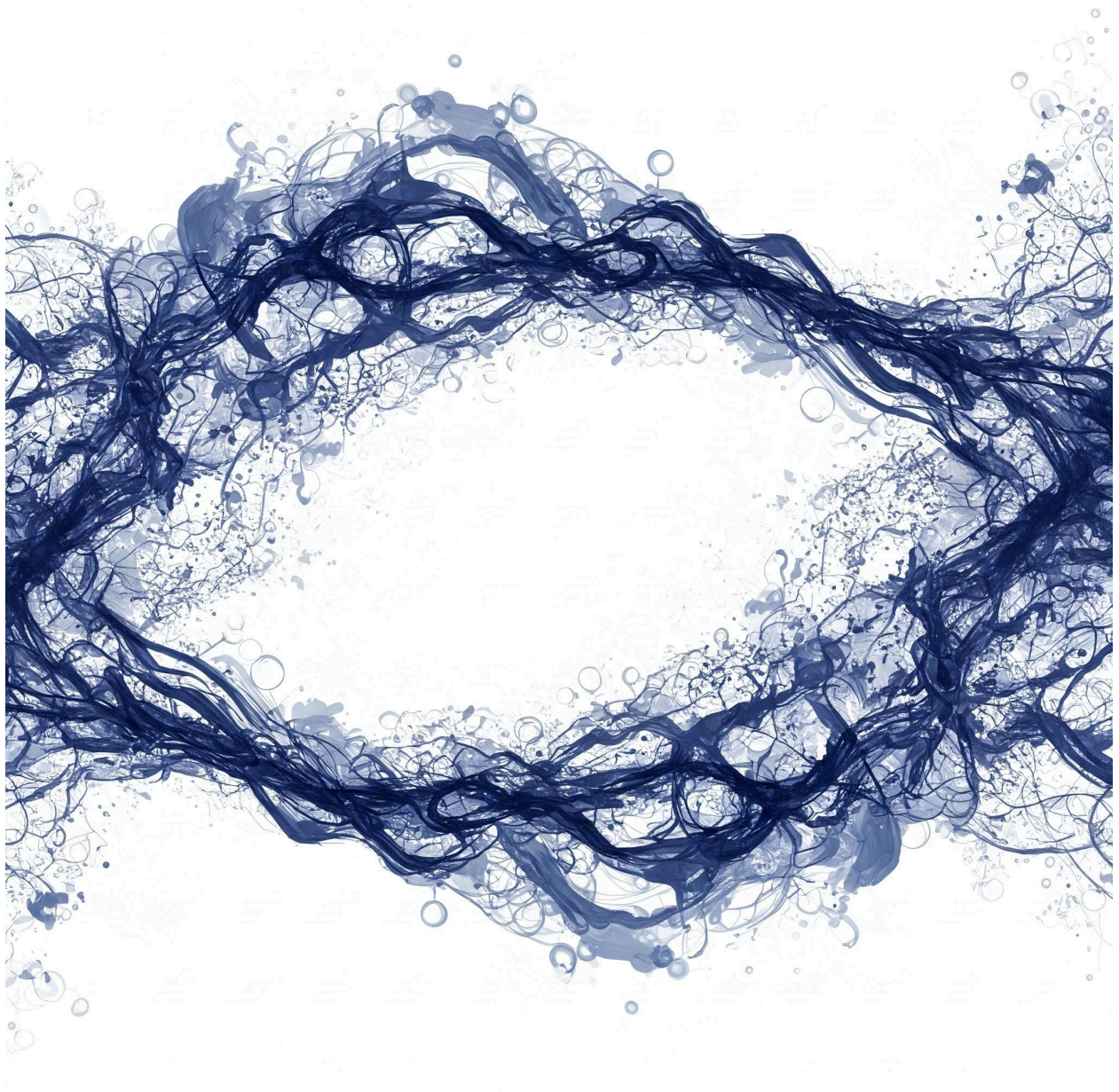
[Glossary](#), 4 pages [final, last read done]

[Index of Terms](#), 11 pages (Generated by Gemini. Reasoning step included as an agentic example) [final, lrd]

[Online contribution - Frequently Asked Questions: Agentic Design Patterns](#)

Pre Print: <https://www.amazon.com/Agentic-Design-Patterns-Hands-Intelligent/dp/3032014018/>

¹ All my royalties will be donated to Save the Children



To my son, Bruno,

who at two years old, brought a new and brilliant light into my life. As I explore the systems that will define our tomorrow, it is the world you will inherit that is foremost in my thoughts.

To my sons, Leonardo and Lorenzo, and my daughter Aurora,

My heart is filled with pride for the women and men you have become and the wonderful world you are building.

This book is about how to build intelligent tools, but it is dedicated to the profound hope that your generation will guide them with wisdom and compassion. The future is incredibly bright, for you and for us all, if we learn to use these powerful technologies to serve humanity and help it progress.

With all my love.

Acknowledgment

I would like to express my sincere gratitude to the many individuals and teams who made this book possible.

First and foremost, I thank Google for adhering to its mission, empowering Googlers, and respecting the opportunity to innovate.

I am grateful to the Office of the CTO for giving me the opportunity to explore new areas, for adhering to its mission of "practical magic," and for its capacity to adapt to new emerging opportunities.

I would like to extend my heartfelt thanks to Will Grannis, our VP, for the trust he puts in people and for being a servant leader. To John Abel, my manager, for encouraging me to pursue my activities and for always providing great guidance with his British acumen. I extend my gratitude to Antoine Larmanjat for our work on LLMs in code, Hann Wang for agent discussions, and Yingchao Huang for time series insights. Thanks to Ashwin Ram for leadership, Massy Mascaro for inspiring work, Jennifer Bennett for technical expertise, Brett Slatkin for engineering, and Eric Schen for stimulating discussions. The OCTO team, especially Scott Penberthy, deserves recognition. Finally, deep appreciation to Patricia Florissi for her inspiring vision of Agents' societal impact.

My appreciation also goes to Marco Argenti for the challenging and motivating vision of agents augmenting the human workforce. My thanks also go to Jim Lanzone and Jordi Ribas for pushing the bar on the relationship between the world of Search and the world of Agents.

I am also indebted to the Cloud AI teams, especially their leader Saurabh Tiwary, for driving the AI organization towards principled progress. Thank you to Salem Salem Haykal, the Area Technical Leader, for being an inspiring colleague. My thanks to Vladimir Vuskovic, co-founder of Google Agentspace, Kate (Katarzyna) Olszewska for our Agentic collaboration on Kaggle Game Arena, and Nate Keating for driving Kaggle with passion, a community that has given so much to AI. My thanks also to Kamelia Aryafa, leading applied AI and ML teams focused on Agentspace and Enterprise NotebookLM, and to Jahn Wooland, a true leader focused on delivering and a personal friend always there to provide advice.

A special thanks to Yingchao Huang for being a brilliant AI engineer with a great career in front of you, Hann Wang for challenging me to return to my interest in Agents after an

initial interest in 1994, and to Lee Boonstra for your amazing work on prompt engineering.

My thanks also go to the 5 Days of GenAI team, including our VP Alison Wagonfeld for the trust put in the team, Anant Nawalgaria for always delivering, and Paige Bailey for her can-do attitude and leadership.

I am also deeply grateful to Mike Styer, Turan Bulmus, and Kanchana Patlolla for helping me ship three Agents at Google I/O 2025. Thank you for your immense work.

I want to express my sincere gratitude to Thomas Kurian for his unwavering leadership, passion, and trust in driving the Cloud and AI initiatives. I also deeply appreciate Emanuel Taropa, whose inspiring "can-do" attitude made him the most exceptional colleague I've encountered at Google, setting a truly profound example. Finally, thanks to Fiona Cicconi for our engaging discussions about Google.

I extend my gratitude to Demis Hassabis, Pushmeet Kohli, and the entire GDM team for their passionate efforts in developing Gemini, AlphaFold, AlphaGo, and AlphaGenome, among other projects, and for their contributions to advancing science for the benefit of society. A special thank you to Yossi Matias for his leadership of Google Research and for consistently offering invaluable advice. I have learned a great deal from you.

A special thanks to Patti Maes, who pioneered the concept of Software Agents in the 90s and remains focused on the question of how computer systems and digital devices might augment people and assist them with issues such as memory, learning, decision making, health, and wellbeing. Your vision back in '91 became a reality today.

I also want to extend my gratitude to Paul Drougas and all the Publisher team at Springer for making this book possible.

I am deeply indebted to the many talented people who helped bring this book to life. My heartfelt thanks go to Marco Fago for his immense contributions, from code and diagrams to reviewing the entire text. I'm also grateful to Mahtab Syed for his coding work and to Ankita Guha for her incredibly detailed feedback on so many chapters. The book was significantly improved by the insightful amendments from Priya Saxena, the careful reviews from Jae Lee, and the dedicated work of Mario da Roza in creating the NotebookLM version. I was fortunate to have a team of expert reviewers for the initial chapters, and I thank Dr. Amita Kapoor, Fatma Tarlaci, PhD, Dr. Alessandro Cornacchia, and Aditya Mandlekar for lending their expertise. My sincere appreciation also goes to Ashley Miller, Amir John, and Palak Kamdar (Vasani) for their unique contributions. For their steadfast support and encouragement, a final, warm thank you is due to Rajat

Jain, Aldo Pahor, Gaurav Verma, Pavithra Sainath, Mariusz Koczwara, Abhijit Kumar, Armstrong Foundjem, Haiming Ran, Udit Patel, and Kaurnakar Kotha.

This project truly would not have been possible without you. All the credit goes to you, and all the mistakes are mine.

All my royalties are donated to Save the Children.

Foreword

The field of artificial intelligence is at a fascinating inflection point. We are moving beyond building models that can simply process information to creating intelligent systems that can reason, plan, and act to achieve complex goals with ambiguous tasks. These "agentic" systems, as this book so aptly describes them, represent the next frontier in AI, and their development is a challenge that excites and inspires us at Google.

"Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems" arrives at the perfect moment to guide us on this journey. The book rightly points out that the power of large language models, the cognitive engines of these agents, must be harnessed with structure and thoughtful design. Just as design patterns revolutionized software engineering by providing a common language and reusable solutions to common problems, the agentic patterns in this book will be foundational for building robust, scalable, and reliable intelligent systems.

The metaphor of a "canvas" for building agentic systems is one that resonates deeply with our work on Google's Vertex AI platform. We strive to provide developers with the most powerful and flexible canvas on which to build the next generation of AI applications. This book provides the practical, hands-on guidance that will empower developers to use that canvas to its full potential. By exploring patterns from prompt chaining and tool use to agent-to-agent collaboration, self-correction, safety and guardrails, this book offers a comprehensive toolkit for any developer looking to build sophisticated AI agents.

The future of AI will be defined by the creativity and ingenuity of developers who can build these intelligent systems. "Agentic Design Patterns" is an indispensable resource that will help to unlock that creativity. It provides the essential knowledge and practical examples to not only understand the "what" and "why" of agentic systems, but also the "how."

I am thrilled to see this book in the hands of the developer community. The patterns and principles within these pages will undoubtedly accelerate the development of innovative and impactful AI applications that will shape our world for years to come.

Saurabh Tiwary

VP & General Manager, CloudAI @ Google

A Thought Leader's Perspective: Power and Responsibility

Of all the technology cycles I've witnessed over the past four decades—from the birth of the personal computer and the web, to the revolutions in mobile and cloud—none has felt quite like this one. For years, the discourse around Artificial Intelligence was a familiar rhythm of hype and disillusionment, the so-called “AI summers” followed by long, cold winters. But this time, something is different. The conversation has palpably shifted. If the last eighteen months were about the engine—the breathtaking, almost vertical ascent of Large Language Models (LLMs)—the next era will be about the car we build around it. It will be about the frameworks that harness this raw power, transforming it from a generator of plausible text into a true agent of action.

I admit, I began as a skeptic. Plausibility, I've found, is often inversely proportional to one's own knowledge of a subject. Early models, for all their fluency, felt like they were operating with a kind of impostor syndrome, optimized for credibility over correctness. But then came the inflection point, a step-change brought about by a new class of "reasoning" models. Suddenly, we weren't just conversing with a statistical machine that predicted the next word in a sequence; we were getting a peek into a nascent form of cognition.

The first time I experimented with one of the new agentic coding tools, I felt that familiar spark of magic. I tasked it with a personal project I'd never found the time for: migrating a charity website from a simple web builder to a proper, modern CI/CD environment. For the next twenty minutes, it went to work, asking clarifying questions, requesting credentials, and providing status updates. It felt less like using a tool and more like collaborating with a junior developer. When it presented me with a fully deployable package, complete with impeccable documentation and unit tests, I was floored.

Of course, it wasn't perfect. It made mistakes. It got stuck. It required my supervision and, crucially, my judgment to steer it back on course. The experience drove home a lesson I've learned the hard way over a long career: you cannot afford to trust blindly. Yet, the process was fascinating. Peeking into its "chain of thought" was like watching a mind at work—messy, non-linear, full of starts, stops, and self-corrections, not unlike our own human reasoning. It wasn't a straight line; it was a random walk toward a solution. Here was the kernel of something new: not just an intelligence that could generate content, but one that could generate a *plan*.

This is the promise of agentic frameworks. It's the difference between a static subway map and a dynamic GPS that reroutes you in real-time. A classic rules-based automaton follows a fixed path; when it encounters an unexpected obstacle, it breaks. An AI agent, powered by a reasoning model, has the potential to observe, adapt, and find another way. It possesses a form of digital common sense that allows it to navigate the countless edge cases of reality. It

represents a shift from simply telling a computer *what* to do, to explaining *why* we need something done and trusting it to figure out the *how*.

As exhilarating as this new frontier is, it brings a profound sense of responsibility, particularly from my vantage point as the CIO of a global financial institution. The stakes are immeasurably high. An agent that makes a mistake while creating a recipe for a "Chicken Salmon Fusion Pie" is a fun anecdote. An agent that makes a mistake while executing a trade, managing risk, or handling client data is a real problem. I've read the disclaimers and the cautionary tales: the web automation agent that, after failing a login, decided to email a member of parliament to complain about login walls. It's a darkly humorous reminder that we are dealing with a technology we don't fully understand.

This is where craft, culture, and a relentless focus on our principles become our essential guide. Our Engineering Tenets are not just words on a page; they are our compass. We must *Build with Purpose*, ensuring that every agent we design starts from a clear understanding of the client problem we are solving. We must *Look Around Corners*, anticipating failure modes and designing systems that are resilient by design. And above all, we must *Inspire Trust*, by being transparent about our methods and accountable for our outcomes.

In an agentic world, these tenets take on new urgency. The hard truth is that you cannot simply overlay these powerful new tools onto messy, inconsistent systems and expect good results. Messy systems plus agents are a recipe for disaster. An AI trained on "garbage" data doesn't just produce garbage-out; it produces plausible, confident garbage that can poison an entire process. Therefore, our first and most critical task is to prepare the ground. We must invest in clean data, consistent metadata, and well-defined APIs. We have to build the modern "interstate system" that allows these agents to operate safely and at high velocity. It is the hard, foundational work of building a programmable enterprise, an "enterprise as software," where our processes are as well-architected as our code.

Ultimately, this journey is not about replacing human ingenuity, but about augmenting it. It demands a new set of skills from all of us: the ability to explain a task with clarity, the wisdom to delegate, and the diligence to verify the quality of the output. It requires us to be humble, to acknowledge what we don't know, and to never stop learning. The pages that follow in this book offer a technical map for building these new frameworks. My hope is that you will use them not just to build what is possible, but to build what is right, what is robust, and what is responsible.

The world is asking every engineer to step up. I am confident we are ready for the challenge.

Enjoy the journey.

Marco Argenti, CIO, Goldman Sachs

Preface

Welcome to "Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems." As we look across the landscape of modern artificial intelligence, we see a clear evolution from simple, reactive programs to sophisticated, autonomous entities capable of understanding context, making decisions, and interacting dynamically with their environment and other systems. These are the intelligent agents and the agentic systems they comprise.

The advent of powerful large language models (LLMs) has provided unprecedented capabilities for understanding and generating human-like content such as text and media, serving as the cognitive engine for many of these agents. However, orchestrating these capabilities into systems that can reliably achieve complex goals requires more than just a powerful model. It requires structure, design, and a thoughtful approach to how the agent perceives, plans, acts, and interacts.

Think of building intelligent systems as creating a complex work of art or engineering on a canvas. This canvas isn't a blank visual space, but rather the underlying infrastructure and frameworks that provide the environment and tools for your agents to exist and operate. It's the foundation upon which you'll build your intelligent application, managing state, communication, tool access, and the flow of logic.

Building effectively on this agentic canvas demands more than just throwing components together. It requires understanding proven techniques – **patterns** – that address common challenges in designing and implementing agent behavior. Just as architectural patterns guide the construction of a building, or design patterns structure software, agentic design patterns provide reusable solutions for the recurring problems you'll face when bringing intelligent agents to life on your chosen canvas.

What are Agentic Systems?

At its core, an agentic system is a computational entity designed to perceive its environment (both digital and potentially physical), make informed decisions based on those perceptions and a set of predefined or learned goals, and execute actions to achieve those goals autonomously. Unlike traditional software, which follows rigid, step-by-step instructions, agents exhibit a degree of flexibility and initiative.

Imagine you need a system to manage customer inquiries. A traditional system might follow a fixed script. An agentic system, however, could perceive the nuances of a customer's query, access knowledge bases, interact with other internal systems (like

order management), potentially ask clarifying questions, and proactively resolve the issue, perhaps even anticipating future needs. These agents operate on the canvas of your application's infrastructure, utilizing the services and data available to them.

Agentic systems are often characterized by features like **autonomy**, allowing them to act without constant human oversight; **proactiveness**, initiating actions towards their goals; and **reactiveness**, responding effectively to changes in their environment. They are fundamentally **goal-oriented**, constantly working towards objectives. A critical capability is **tool use**, enabling them to interact with external APIs, databases, or services – effectively reaching out beyond their immediate canvas. They possess **memory**, retain information across interactions, and can engage in **communication** with users, other systems, or even other agents operating on the same or connected canvases.

Effectively realizing these characteristics introduces significant complexity. How does the agent maintain state across multiple steps on its canvas? How does it decide *when* and *how* to use a tool? How is communication between different agents managed? How do you build resilience into the system to handle unexpected outcomes or errors?

Why Patterns Matter in Agent Development

This complexity is precisely why agentic design patterns are indispensable. They are not rigid rules, but rather battle-tested templates or blueprints that offer proven approaches to standard design and implementation challenges in the agentic domain. By recognizing and applying these design patterns, you gain access to solutions that enhance the structure, maintainability, reliability, and efficiency of the agents you build on your canvas.

Using design patterns helps you avoid reinventing fundamental solutions for tasks like managing conversational flow, integrating external capabilities, or coordinating multiple agent actions. They provide a common language and structure that makes your agent's logic clearer and easier for others (and yourself in the future) to understand and maintain. Implementing patterns designed for error handling or state management directly contributes to building more robust and reliable systems. Leveraging these established approaches accelerates your development process, allowing you to focus on the unique aspects of your application rather than the foundational mechanics of agent behavior.

This book extracts 21 key design patterns that represent fundamental building blocks and techniques for constructing sophisticated agents on various technical canvases.

Understanding and applying these patterns will significantly elevate your ability to design and implement intelligent systems effectively.

Overview of the Book and How to Use It

This book, "Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems," is crafted to be a practical and accessible resource. Its primary focus is on clearly explaining each agentic pattern and providing concrete, runnable code examples to demonstrate its implementation. Across 21 dedicated chapters, we will explore a diverse range of design patterns, from foundational concepts like structuring sequential operations (Prompt Chaining) and external interaction (Tool Use) to more advanced topics like collaborative work (Multi-Agent Collaboration) and self-improvement (Self-Correction).

The book is organized chapter by chapter, with each chapter delving into a single agentic pattern. Within each chapter, you will find:

- A detailed **Pattern Overview** providing a clear explanation of the pattern and its role in agentic design.
- A section on **Practical Applications & Use Cases** illustrating real-world scenarios where the pattern is invaluable and the benefits it brings.
- A **Hands-On Code Example** offering practical, runnable code that demonstrates the pattern's implementation using prominent agent development frameworks. This is where you'll see how to apply the pattern within the context of a technical canvas.
- **Key Takeaways** summarizing the most crucial points for quick review.
- **References** for further exploration, providing resources for deeper learning on the pattern and related concepts.

While the chapters are ordered to build concepts progressively, feel free to use the book as a reference, jumping to chapters that address specific challenges you face in your own agent development projects. The appendices provide a comprehensive look at advanced prompting techniques, principles for applying AI agents in real-world environments, and an overview of essential agentic frameworks. To complement this, practical online-only tutorials are included, offering step-by-step guidance on building agents with specific platforms like AgentSpace and for the command-line interface. The emphasis throughout is on practical application; we strongly encourage you to run the code examples, experiment with them, and adapt them to build your own intelligent systems on your chosen canvas.

A great question I hear is, 'With AI changing so fast, why write a book that could be quickly outdated?' My motivation was actually the opposite. It's precisely because things are moving so quickly that we need to step back and identify the underlying principles that are solidifying. Patterns like RAG, Reflection, Routing, Memory and the others I discuss, are becoming fundamental building blocks. This book is an invitation to reflect on these core ideas, which provide the foundation we need to build upon. Humans need these reflection moments on foundation patterns

Introduction to the Frameworks Used

To provide a tangible "canvas" for our code examples (see also Appendix), we will primarily utilize three prominent agent development frameworks. **LangChain**, along with its stateful extension **LangGraph**, provides a flexible way to chain together language models and other components, offering a robust canvas for building complex sequences and graphs of operations. **Crew AI** provides a structured framework specifically designed for orchestrating multiple AI agents, roles, and tasks, acting as a canvas particularly well-suited for collaborative agent systems. The **Google Agent Developer Kit (Google ADK)** offers tools and components for building, evaluating, and deploying agents, providing another valuable canvas, often integrated with Google's AI infrastructure.

These frameworks represent different facets of the agent development canvas, each with its strengths. By showing examples across these tools, you will gain a broader understanding of how the patterns can be applied regardless of the specific technical environment you choose for your agentic systems. The examples are designed to clearly illustrate the pattern's core logic and its implementation on the framework's canvas, focusing on clarity and practicality.

By the end of this book, you will not only understand the fundamental concepts behind 21 essential agentic patterns but also possess the practical knowledge and code examples to apply them effectively, enabling you to build more intelligent, capable, and autonomous systems on your chosen development canvas. Let's begin this hands-on journey!

What makes an AI system an Agent?

In simple terms, an **AI agent** is a system designed to perceive its environment and take actions to achieve a specific goal. It's an evolution from a standard Large Language Model (LLM), enhanced with the abilities to plan, use tools, and interact with its surroundings. Think of an Agentic AI as a smart assistant that learns on the job. It follows a simple, five-step loop to get things done (see Fig.1):

1. **Get the Mission:** You give it a goal, like "organize my schedule."
2. **Scan the Scene:** It gathers all the necessary information—reading emails, checking calendars, and accessing contacts—to understand what's happening.
3. **Think It Through:** It devises a plan of action by considering the optimal approach to achieve the goal.
4. **Take Action:** It executes the plan by sending invitations, scheduling meetings, and updating your calendar.
5. **Learn and Get Better:** It observes successful outcomes and adapts accordingly. For example, if a meeting is rescheduled, the system learns from this event to enhance its future performance.

Agentic AI Problem-Solving Process

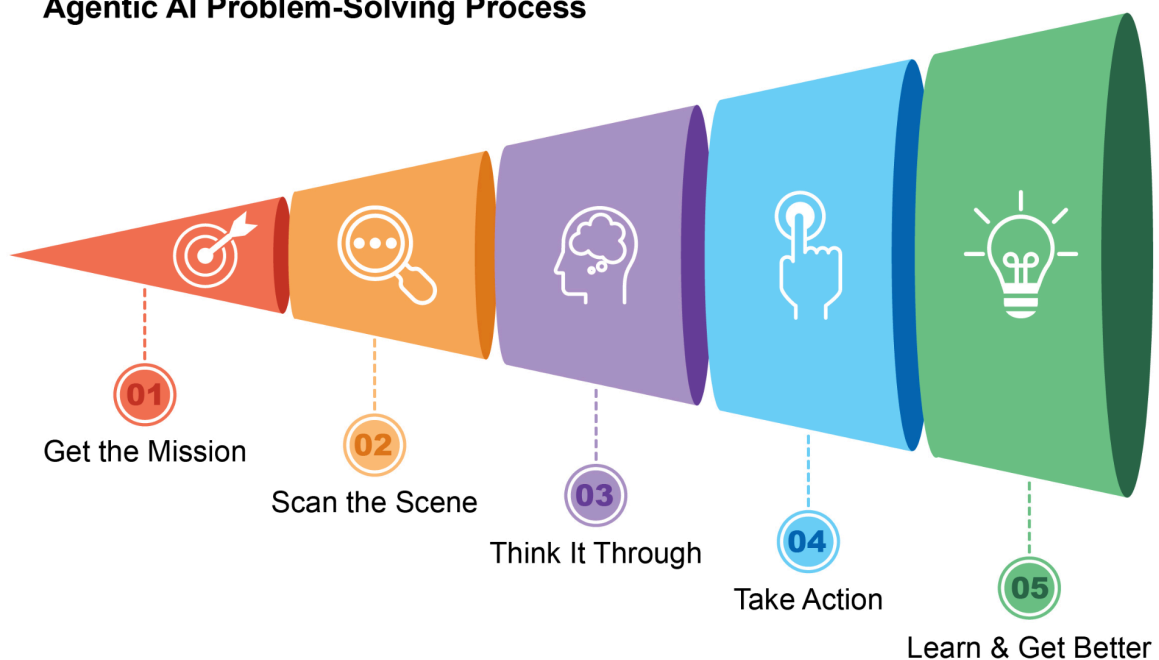


Fig.1: Agentic AI functions as an intelligent assistant, continuously learning through experience. It operates via a straightforward five-step loop to accomplish tasks.

Agents are becoming increasingly popular at a stunning pace. According to recent studies, a majority of large IT companies are actively using these agents, and a fifth of them just started within the past year. The financial markets are also taking notice. By the end of 2024, AI agent startups had raised more than \$2 billion, and the market was valued at \$5.2 billion. It's expected to explode to nearly \$200 billion in value by 2034. In short, all signs point to AI agents playing a massive role in our future economy.

In just two years, the AI paradigm has shifted dramatically, moving from simple automation to sophisticated, autonomous systems (see Fig. 2). Initially, workflows relied on basic prompts and triggers to process data with LLMs. This evolved with Retrieval-Augmented Generation (RAG), which enhanced reliability by grounding models on factual information. We then saw the development of individual AI Agents capable of using various tools. Today, we are entering the era of Agentic AI, where a team of specialized agents works in concert to achieve complex goals, marking a significant leap in AI's collaborative power.

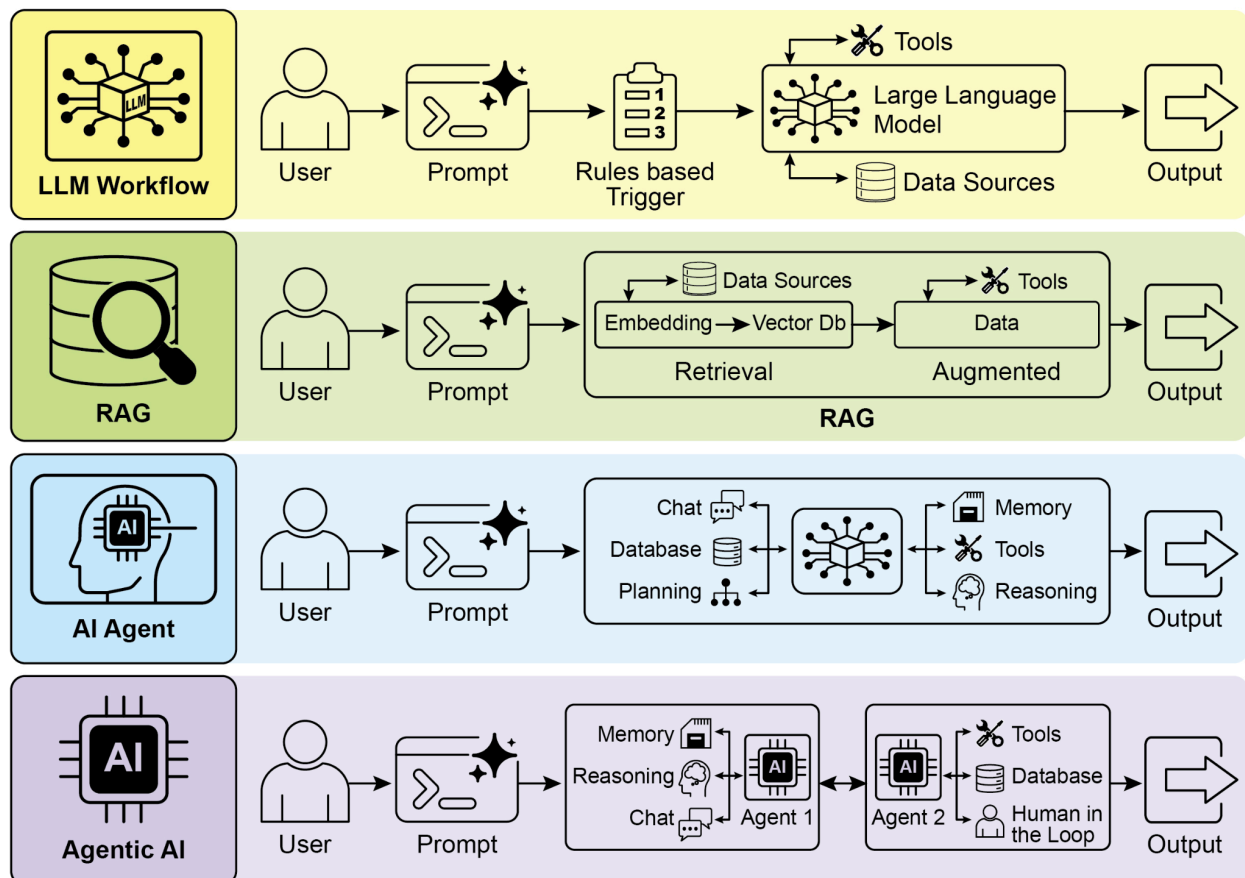


Fig 2.: Transitioning from LLMs to RAG, then to Agentic RAG, and finally to Agentic AI.

The intent of this book is to discuss the design patterns of how specialized agents can work in concert and collaborate to achieve complex goals, and you will see one paradigm of collaboration and interaction in each chapter.

Before doing that, let's examine examples that span the range of agent complexity (see Fig. 3).

Level 0: The Core Reasoning Engine

While an LLM is not an agent in itself, it can serve as the reasoning core of a basic agentic system. In a 'Level 0' configuration, the LLM operates without tools, memory, or environment interaction, responding solely based on its pretrained knowledge. Its strength lies in leveraging its extensive training data to explain established concepts. The trade-off for this powerful internal reasoning is a complete lack of current-event awareness. For instance, it would be unable to name the 2025 Oscar winner for "Best Picture" if that information is outside its pre-trained knowledge.

Level 1: The Connected Problem-Solver

At this level, the LLM becomes a functional agent by connecting to and utilizing external tools. Its problem-solving is no longer limited to its pre-trained knowledge. Instead, it can execute a sequence of actions to gather and process information from sources like the internet (via search) or databases (via Retrieval Augmented Generation, or RAG). For detailed information, refer to Chapter 14.

For instance, to find new TV shows, the agent recognizes the need for current information, uses a search tool to find it, and then synthesizes the results. Crucially, it can also use specialized tools for higher accuracy, such as calling a financial API to get the live stock price for AAPL. This ability to interact with the outside world across multiple steps is the core capability of a Level 1 agent.

Level 2: The Strategic Problem-Solver

At this level, an agent's capabilities expand significantly, encompassing strategic planning, proactive assistance, and self-improvement, with prompt engineering and context engineering as core enabling skills.

First, the agent moves beyond single-tool use to tackle complex, multi-part problems through strategic problem-solving. As it executes a sequence of actions, it actively

performs context engineering: the strategic process of selecting, packaging, and managing the most relevant information for each step. For example, to find a coffee shop between two locations, it first uses a mapping tool. It then engineers this output, curating a short, focused context—perhaps just a list of street names—to feed into a local search tool, preventing cognitive overload and ensuring the second step is efficient and accurate. To achieve maximum accuracy from an AI, it must be given a short, focused, and powerful context. Context engineering is the discipline that accomplishes this by strategically selecting, packaging, and managing the most critical information from all available sources. It effectively curates the model's limited attention to prevent overload and ensure high-quality, efficient performance on any given task. For detailed information, refer to the Appendix A.

This level leads to proactive and continuous operation. A travel assistant linked to your email demonstrates this by engineering the context from a verbose flight confirmation email; it selects only the key details (flight numbers, dates, locations) to package for subsequent tool calls to your calendar and a weather API.

In specialized fields like software engineering, the agent manages an entire workflow by applying this discipline. When assigned a bug report, it reads the report and accesses the codebase, then strategically engineers these large sources of information into a potent, focused context that allows it to efficiently write, test, and submit the correct code patch.

Finally, the agent achieves self-improvement by refining its own context engineering processes. When it asks for feedback on how a prompt could have been improved, it is learning how to better curate its initial inputs. This allows it to automatically improve how it packages information for future tasks, creating a powerful, automated feedback loop that increases its accuracy and efficiency over time. For detailed information, refer to Chapter 17.

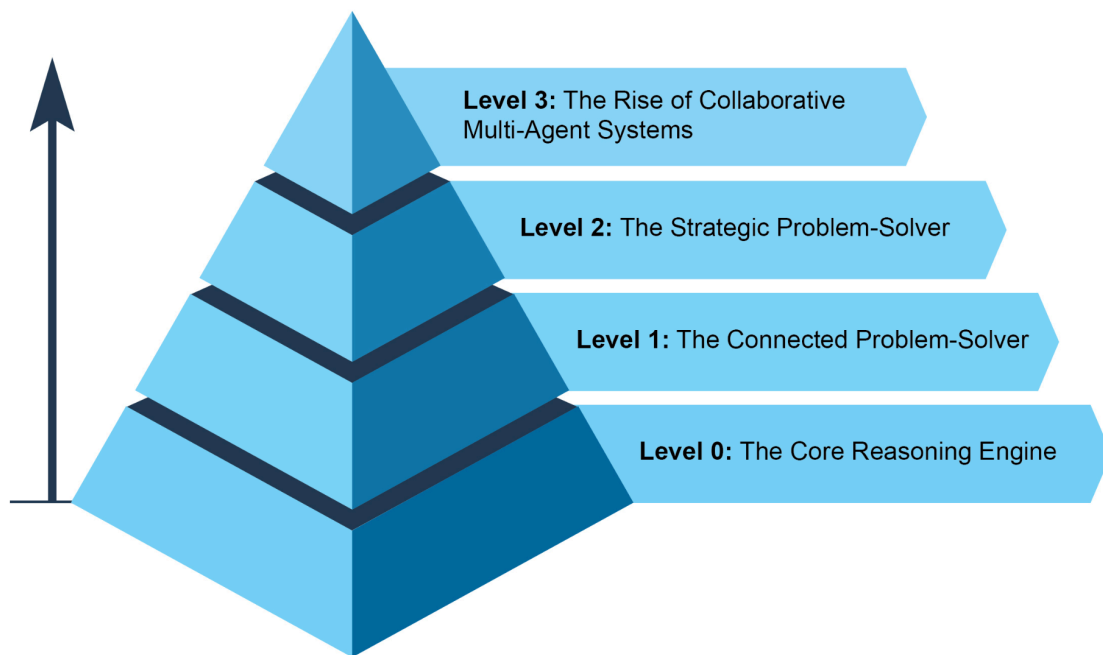


Fig. 3: Various instances demonstrating the spectrum of agent complexity.

Level 3: The Rise of Collaborative Multi-Agent Systems

At Level 3, we see a significant paradigm shift in AI development, moving away from the pursuit of a single, all-powerful super-agent and towards the rise of sophisticated, collaborative multi-agent systems. In essence, this approach recognizes that complex challenges are often best solved not by a single generalist, but by a team of specialists working in concert. This model directly mirrors the structure of a human organization, where different departments are assigned specific roles and collaborate to tackle multi-faceted objectives. The collective strength of such a system lies in this division of labor and the synergy created through coordinated effort. For detailed information, refer to Chapter 7.

To bring this concept to life, consider the intricate workflow of launching a new product. Rather than one agent attempting to handle every aspect, a "Project Manager" agent could serve as the central coordinator. This manager would orchestrate the entire process by delegating tasks to other specialized agents: a "Market Research" agent to gather consumer data, a "Product Design" agent to develop concepts, and a "Marketing" agent to craft promotional materials. The key to their success would be the seamless communication and information sharing between them, ensuring all individual efforts align to achieve the collective goal.

While this vision of autonomous, team-based automation is already being developed, it's important to acknowledge the current hurdles. The effectiveness of such multi-agent systems is presently constrained by the reasoning limitations of LLMs they are using. Furthermore, their ability to genuinely learn from one another and improve as a cohesive unit is still in its early stages. Overcoming these technological bottlenecks is the critical next step, and doing so will unlock the profound promise of this level: the ability to automate entire business workflows from start to finish.

The Future of Agents: Top 5 Hypotheses

AI agent development is progressing at an unprecedented pace across domains such as software automation, scientific research, and customer service among others. While current systems are impressive, they are just the beginning. The next wave of innovation will likely focus on making agents more reliable, collaborative, and deeply integrated into our lives. Here are five leading hypotheses for what's next (see Fig. 4).

Hypothesis 1: The Emergence of the Generalist Agent

The first hypothesis is that AI agents will evolve from narrow specialists into true generalists capable of managing complex, ambiguous, and long-term goals with high reliability. For instance, you could give an agent a simple prompt like, "Plan my company's offsite retreat for 30 people in Lisbon next quarter." The agent would then manage the entire project for weeks, handling everything from budget approvals and flight negotiations to venue selection and creating a detailed itinerary from employee feedback, all while providing regular updates. Achieving this level of autonomy will require fundamental breakthroughs in AI reasoning, memory, and near-perfect reliability. An alternative, yet not mutually exclusive, approach is the rise of Small Language Models (SLMs). This "Lego-like" concept involves composing systems from small, specialized expert agents rather than scaling up a single monolithic model. This method promises systems that are cheaper, faster to debug, and easier to deploy. Ultimately, the development of large generalist models and the composition of smaller specialized ones are both plausible paths forward, and they could even complement each other.

Hypothesis 2: Deep Personalization and Proactive Goal Discovery

The second hypothesis posits that agents will become deeply personalised and proactive partners. We are witnessing the emergence of a new class of agent: the proactive partner. By learning from your unique patterns and goals, these systems are beginning to shift from just following orders to anticipating your needs. AI systems

operate as agents when they move beyond simply responding to chats or instructions. They initiate and execute tasks on behalf of the user, actively collaborating in the process. This moves beyond simple task execution into the realm of proactive goal discovery.

For instance, if you're exploring sustainable energy, the agent might identify your latent goal and proactively support it by suggesting courses or summarizing research. While these systems are still developing, their trajectory is clear. They will become increasingly proactive, learning to take initiative on your behalf when highly confident that the action will be helpful. Ultimately, the agent becomes an indispensable ally, helping you discover and achieve ambitions you have yet to fully articulate.

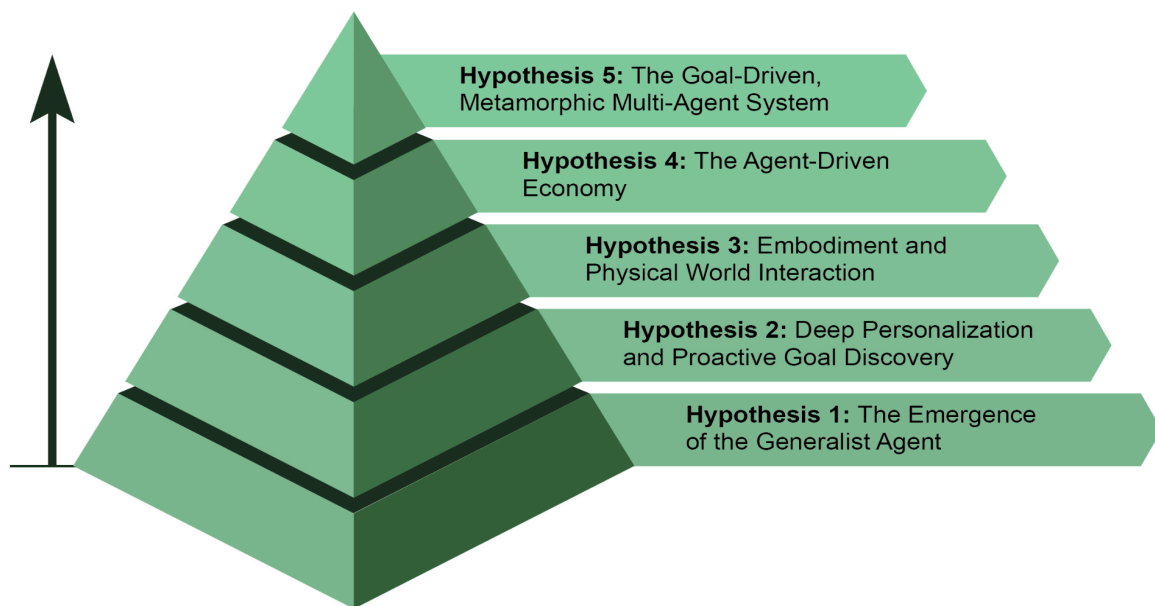


Fig. 4: Five hypotheses about the future of agents

Hypothesis 3: Embodiment and Physical World Interaction

This hypothesis foresees agents breaking free from their purely digital confines to operate in the physical world. By integrating agentic AI with robotics, we will see the rise of "embodied agents." Instead of just booking a handyman, you might ask your home agent to fix a leaky tap. The agent would use its vision sensors to perceive the problem,

access a library of plumbing knowledge to formulate a plan, and then control its robotic manipulators with precision to perform the repair. This would represent a monumental step, bridging the gap between digital intelligence and physical action, and transforming everything from manufacturing and logistics to elder care and home maintenance.

Hypothesis 4: The Agent-Driven Economy

The fourth hypothesis is that highly autonomous agents will become active participants in the economy, creating new markets and business models. We could see agents acting as independent economic entities, tasked with maximising a specific outcome, such as profit. An entrepreneur could launch an agent to run an entire e-commerce business. The agent would identify trending products by analysing social media, generate marketing copy and visuals, manage supply chain logistics by interacting with other automated systems, and dynamically adjust pricing based on real-time demand. This shift would create a new, hyper-efficient "agent economy" operating at a speed and scale impossible for humans to manage directly.

Hypothesis 5: The Goal-Driven, Metamorphic Multi-Agent System

This hypothesis posits the emergence of intelligent systems that operate not from explicit programming, but from a declared goal. The user simply states the desired outcome, and the system autonomously figures out how to achieve it. This marks a fundamental shift towards metamorphic multi-agent systems capable of true self-improvement at both the individual and collective levels.

This system would be a dynamic entity, not a single agent. It would have the ability to analyze its own performance and modify the topology of its multi-agent workforce, creating, duplicating, or removing agents as needed to form the most effective team for the task at hand. This evolution happens at multiple levels:

- Architectural Modification: At the deepest level, individual agents can rewrite their own source code and re-architect their internal structures for higher efficiency, as in the original hypothesis.
- Instructional Modification: At a higher level, the system continuously performs automatic prompt engineering and context engineering. It refines the instructions and information given to each agent, ensuring they are operating with optimal guidance without any human intervention.

For instance, an entrepreneur would simply declare the intent: "Launch a successful e-commerce business selling artisanal coffee." The system, without further programming, would spring into action. It might initially spawn a "Market Research" agent and a "Branding" agent. Based on the initial findings, it could decide to remove

the branding agent and spawn three new specialized agents: a "Logo Design" agent, a "Webstore Platform" agent, and a "Supply Chain" agent. It would constantly tune their internal prompts for better performance. If the webstore agent becomes a bottleneck, the system might duplicate it into three parallel agents to work on different parts of the site, effectively re-architecting its own structure on the fly to best achieve the declared goal.

Conclusion

In essence, an AI agent represents a significant leap from traditional models, functioning as an autonomous system that perceives, plans, and acts to achieve specific goals. The evolution of this technology is advancing from single, tool-using agents to complex, collaborative multi-agent systems that tackle multifaceted objectives. Future hypotheses predict the emergence of generalist, personalized, and even physically embodied agents that will become active participants in the economy. This ongoing development signals a major paradigm shift towards self-improving, goal-driven systems poised to automate entire workflows and fundamentally redefine our relationship with technology.

References

1. Cloudera, Inc. (April 2025), 96% of enterprises are increasing their use of AI agents. <https://www.cloudera.com/about/news-and-blogs/press-releases/2025-04-16-96-percent-of-enterprises-are-expanding-use-of-ai-agents-according-to-latest-data-from-cloudera.html>
2. Autonomous generative AI agents: <https://www.deloitte.com/us/en/insights/industry/technology/technology-media-and-telecom-predictions/2025/autonomous-generative-ai-agents-still-under-development.html>
3. Market.us. Global Agentic AI Market Size, Trends and Forecast 2025–2034. <https://market.us/report/agentic-ai-market/>

Chapter 1: Prompt Chaining

Prompt Chaining Pattern Overview

Prompt chaining, sometimes referred to as Pipeline pattern, represents a powerful paradigm for handling intricate tasks when leveraging large language models (LLMs). Rather than expecting an LLM to solve a complex problem in a single, monolithic step, prompt chaining advocates for a divide-and-conquer strategy. The core idea is to break down the original, daunting problem into a sequence of smaller, more manageable sub-problems. Each sub-problem is addressed individually through a specifically designed prompt, and the output generated from one prompt is strategically fed as input into the subsequent prompt in the chain.

This sequential processing technique inherently introduces modularity and clarity into the interaction with LLMs. By decomposing a complex task, it becomes easier to understand and debug each individual step, making the overall process more robust and interpretable. Each step in the chain can be meticulously crafted and optimized to focus on a specific aspect of the larger problem, leading to more accurate and focused outputs.

The output of one step acting as the input for the next is crucial. This passing of information establishes a dependency chain, hence the name, where the context and results of previous operations guide the subsequent processing. This allows the LLM to build on its previous work, refine its understanding, and progressively move closer to the desired solution.

Furthermore, prompt chaining is not just about breaking down problems; it also enables the integration of external knowledge and tools. At each step, the LLM can be instructed to interact with external systems, APIs, or databases, enriching its knowledge and abilities beyond its internal training data. This capability dramatically expands the potential of LLMs, allowing them to function not just as isolated models but as integral components of broader, more intelligent systems.

The significance of prompt chaining extends beyond simple problem-solving. It serves as a foundational technique for building sophisticated AI agents. These agents can utilize prompt chains to autonomously plan, reason, and act in dynamic environments. By strategically structuring the sequence of prompts, an agent can engage in tasks requiring multi-step reasoning, planning, and decision-making. Such agent workflows can mimic human thought processes more closely, allowing for more natural and effective interactions with complex domains and systems.

Limitations of single prompts: For multifaceted tasks, using a single, complex prompt for an LLM can be inefficient, causing the model to struggle with constraints and instructions, potentially leading to instruction neglect where parts of the prompt are overlooked, contextual drift where the model loses track of the initial context, error propagation where early errors amplify, prompts which require a longer context window where the model gets insufficient information to respond back and hallucination where the cognitive load increases the chance of incorrect information. For example, a query asking to analyze a market research report, summarize findings, identify trends with data points, and draft an email risks failure as the model might summarize well but fail to extract data or draft an email properly.

Enhanced Reliability Through Sequential Decomposition: Prompt chaining addresses these challenges by breaking the complex task into a focused, sequential workflow, which significantly improves reliability and control. Given the example above, a pipeline or chained approach can be described as follows:

1. Initial Prompt (Summarization): "Summarize the key findings of the following market research report: [text]." The model's sole focus is summarization, increasing the accuracy of this initial step.
2. Second Prompt (Trend Identification): "Using the summary, identify the top three emerging trends and extract the specific data points that support each trend: [output from step 1]." This prompt is now more constrained and builds directly upon a validated output.
3. Third Prompt (Email Composition): "Draft a concise email to the marketing team that outlines the following trends and their supporting data: [output from step 2]."

This decomposition allows for more granular control over the process. Each step is simpler and less ambiguous, which reduces the cognitive load on the model and leads to a more accurate and reliable final output. This modularity is analogous to a computational pipeline where each function performs a specific operation before passing its result to the next. To ensure an accurate response for each specific task, the model can be assigned a distinct role at every stage. For example, in the given scenario, the initial prompt could be designated as "Market Analyst," the subsequent prompt as "Trade Analyst," and the third prompt as "Expert Documentation Writer," and so forth.

The Role of Structured Output: The reliability of a prompt chain is highly dependent on the integrity of the data passed between steps. If the output of one prompt is ambiguous or poorly formatted, the subsequent prompt may fail due to faulty input. To mitigate this, specifying a structured output format, such as JSON or XML, is crucial.

For example, the output from the trend identification step could be formatted as a JSON object:

```
{
  "trends": [
    {
      "trend_name": "AI-Powered Personalization",
      "supporting_data": "73% of consumers prefer to do business with brands that use personal information to make their shopping experiences more relevant."
    },
    {
      "trend_name": "Sustainable and Ethical Brands",
      "supporting_data": "Sales of products with ESG-related claims grew 28% over the last five years, compared to 20% for products without."
    }
  ]
}
```

This structured format ensures that the data is machine-readable and can be precisely parsed and inserted into the next prompt without ambiguity. This practice minimizes errors that can arise from interpreting natural language and is a key component in building robust, multi-step LLM-based systems.

Practical Applications & Use Cases

Prompt chaining is a versatile pattern applicable in a wide range of scenarios when building agentic systems. Its core utility lies in breaking down complex problems into sequential, manageable steps. Here are several practical applications and use cases:

1. Information Processing Workflows: Many tasks involve processing raw information through multiple transformations. For instance, summarizing a document, extracting key entities, and then using those entities to query a database or generate a report. A prompt chain could look like:

- Prompt 1: Extract text content from a given URL or document.
- Prompt 2: Summarize the cleaned text.
- Prompt 3: Extract specific entities (e.g., names, dates, locations) from the summary or original text.
- Prompt 4: Use the entities to search an internal knowledge base.
- Prompt 5: Generate a final report incorporating the summary, entities, and search results.

This methodology is applied in domains such as automated content analysis, the development of AI-driven research assistants, and complex report generation.

2. Complex Query Answering: Answering complex questions that require multiple steps of reasoning or information retrieval is a prime use case. For example, "What were the main causes of the stock market crash in 1929, and how did government policy respond?"

- Prompt 1: Identify the core sub-questions in the user's query (causes of crash, government response).
- Prompt 2: Research or retrieve information specifically about the causes of the 1929 crash.
- Prompt 3: Research or retrieve information specifically about the government's policy response to the 1929 stock market crash.
- Prompt 4: Synthesize the information from steps 2 and 3 into a coherent answer to the original query.

This sequential processing methodology is integral to developing AI systems capable of multi-step inference and information synthesis. Such systems are required when a query cannot be answered from a single data point but instead necessitates a series of logical steps or the integration of information from diverse sources.

For example, an automated research agent designed to generate a comprehensive report on a specific topic executes a hybrid computational workflow. Initially, the system retrieves numerous relevant articles. The subsequent task of extracting key information from each article can be performed concurrently for each source. This stage is well-suited for parallel processing, where independent sub-tasks are run simultaneously to maximize efficiency.

However, once the individual extractions are complete, the process becomes inherently sequential. The system must first collate the extracted data, then synthesize it into a coherent draft, and finally review and refine this draft to produce a final report. Each of these later stages is logically dependent on the successful completion of the preceding one. This is where prompt chaining is applied: the collated data serves as the input for the synthesis prompt, and the resulting synthesized text becomes the input for the final review prompt. Therefore, complex operations frequently combine parallel processing for independent data gathering with prompt chaining for the dependent steps of synthesis and refinement.

3. Data Extraction and Transformation: The conversion of unstructured text into a structured format is typically achieved through an iterative process, requiring sequential modifications to improve the accuracy and completeness of the output.

- Prompt 1: Attempt to extract specific fields (e.g., name, address, amount) from an invoice document.

- Processing: Check if all required fields were extracted and if they meet format requirements.
- Prompt 2 (Conditional): If fields are missing or malformed, craft a new prompt asking the model to specifically find the missing/malformed information, perhaps providing context from the failed attempt.
- Processing: Validate the results again. Repeat if necessary.
- Output: Provide the extracted, validated structured data.

This sequential processing methodology is particularly applicable to data extraction and analysis from unstructured sources like forms, invoices, or emails. For example, solving complex Optical Character Recognition (OCR) problems, such as processing a PDF form, is more effectively handled through a decomposed, multi-step approach.

Initially, a large language model is employed to perform the primary text extraction from the document image. Following this, the model processes the raw output to normalize the data, a step where it might convert numeric text, such as "one thousand and fifty," into its numerical equivalent, 1050. A significant challenge for LLMs is performing precise mathematical calculations. Therefore, in a subsequent step, the system can delegate any required arithmetic operations to an external calculator tool. The LLM identifies the necessary calculation, feeds the normalized numbers to the tool, and then incorporates the precise result. This chained sequence of text extraction, data normalization, and external tool use achieves a final, accurate result that is often difficult to obtain reliably from a single LLM query.

4. Content Generation Workflows: The composition of complex content is a procedural task that is typically decomposed into distinct phases, including initial ideation, structural outlining, drafting, and subsequent revision

- Prompt 1: Generate 5 topic ideas based on a user's general interest.
- Processing: Allow the user to select one idea or automatically choose the best one.
- Prompt 2: Based on the selected topic, generate a detailed outline.
- Prompt 3: Write a draft section based on the first point in the outline.
- Prompt 4: Write a draft section based on the second point in the outline, providing the previous section for context. Continue this for all outline points.
- Prompt 5: Review and refine the complete draft for coherence, tone, and grammar.

This methodology is employed for a range of natural language generation tasks, including the automated composition of creative narratives, technical documentation, and other forms of structured textual content.

5. Conversational Agents with State: Although comprehensive state management architectures employ methods more complex than sequential linking, prompt chaining provides a foundational mechanism for preserving conversational continuity. This technique maintains

context by constructing each conversational turn as a new prompt that systematically incorporates information or extracted entities from preceding interactions in the dialogue sequence.

- Prompt 1: Process User Utterance 1, identify intent and key entities.
- Processing: Update conversation state with intent and entities.
- Prompt 2: Based on current state, generate a response and/or identify the next required piece of information.
- Repeat for subsequent turns, with each new user utterance initiating a chain that leverages the accumulating conversation history (state).

This principle is fundamental to the development of conversational agents, enabling them to maintain context and coherence across extended, multi-turn dialogues. By preserving the conversational history, the system can understand and appropriately respond to user inputs that depend on previously exchanged information.

6. Code Generation and Refinement: The generation of functional code is typically a multi-stage process, requiring a problem to be decomposed into a sequence of discrete logical operations that are executed progressively

- Prompt 1: Understand the user's request for a code function. Generate pseudocode or an outline.
- Prompt 2: Write the initial code draft based on the outline.
- Prompt 3: Identify potential errors or areas for improvement in the code (perhaps using a static analysis tool or another LLM call).
- Prompt 4: Rewrite or refine the code based on the identified issues.
- Prompt 5: Add documentation or test cases.

In applications such as AI-assisted software development, the utility of prompt chaining stems from its capacity to decompose complex coding tasks into a series of manageable sub-problems. This modular structure reduces the operational complexity for the large language model at each step. Critically, this approach also allows for the insertion of deterministic logic between model calls, enabling intermediate data processing, output validation, and conditional branching within the workflow. By this method, a single, multifaceted request that could otherwise lead to unreliable or incomplete results is converted into a structured sequence of operations managed by an underlying execution framework.

7. Multimodal and multi-step reasoning: Analyzing datasets with diverse modalities necessitates breaking down the problem into smaller, prompt-based tasks. For example, interpreting an image that contains a picture with embedded text, labels highlighting specific text segments, and tabular data explaining each label, requires such an approach.

- Prompt 1: Extract and comprehend the text from the user's image request.
- Prompt 2: Link the extracted image text with its corresponding labels.
- Prompt 3: Interpret the gathered information using a table to determine the required output.

Hands-On Code Example

Implementing prompt chaining ranges from direct, sequential function calls within a script to the utilization of specialized frameworks designed to manage control flow, state, and component integration. Frameworks such as LangChain, LangGraph, Crew AI, and the Google Agent Development Kit (ADK) offer structured environments for constructing and executing these multi-step processes, which is particularly advantageous for complex architectures.

For the purpose of demonstration, LangChain and LangGraph are suitable choices as their core APIs are explicitly designed for composing chains and graphs of operations. LangChain provides foundational abstractions for linear sequences, while LangGraph extends these capabilities to support stateful and cyclical computations, which are necessary for implementing more sophisticated agentic behaviors. This example will focus on a fundamental linear sequence.

The following code implements a two-step prompt chain that functions as a data processing pipeline. The initial stage is designed to parse unstructured text and extract specific information. The subsequent stage then receives this extracted output and transforms it into a structured data format.

To replicate this procedure, the required libraries must first be installed. This can be accomplished using the following command:

```
pip install langchain langchain-community langchain-openai langgraph
```

Note that langchain-openai can be substituted with the appropriate package for a different model provider. Subsequently, the execution environment must be configured with the necessary API credentials for the selected language model provider, such as OpenAI, Google Gemini, or Anthropic.

```
import os
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# For better security, load environment variables from a .env file
# from dotenv import load_dotenv
```

```

# load_dotenv()
# Make sure your OPENAI_API_KEY is set in the .env file

# Initialize the Language Model (using ChatOpenAI is recommended)
llm = ChatOpenAI(temperature=0)

# --- Prompt 1: Extract Information ---
prompt_extract = ChatPromptTemplate.from_template(
    "Extract the technical specifications from the following
text:\n\n{text_input}"
)

# --- Prompt 2: Transform to JSON ---
prompt_transform = ChatPromptTemplate.from_template(
    "Transform the following specifications into a JSON object with
'cpu', 'memory', and 'storage' as keys:\n\n{specifications}"
)

# --- Build the Chain using LCEL ---
# The StrOutputParser() converts the LLM's message output to a simple
string.
extraction_chain = prompt_extract | llm | StrOutputParser()

# The full chain passes the output of the extraction chain into the
'specifications'
# variable for the transformation prompt.
full_chain = (
    {"specifications": extraction_chain}
    | prompt_transform
    | llm
    | StrOutputParser()
)

# --- Run the Chain ---
input_text = "The new laptop model features a 3.5 GHz octa-core
processor, 16GB of RAM, and a 1TB NVMe SSD."

# Execute the chain with the input text dictionary.
final_result = full_chain.invoke({"text_input": input_text})

print("\n--- Final JSON Output ---")
print(final_result)

```

This Python code demonstrates how to use the LangChain library to process text. It utilizes two separate prompts: one to extract technical specifications from an input string and another to format these specifications into a JSON object. The ChatOpenAI model is employed for language model interactions, and the StrOutputParser ensures the output is in a usable string

format. The LangChain Expression Language (LCEL) is used to elegantly chain these prompts and the language model together. The first chain, `extraction_chain`, extracts the specifications. The `full_chain` then takes the output of the extraction and uses it as input for the transformation prompt. A sample input text describing a laptop is provided. The `full_chain` is invoked with this text, processing it through both steps. The final result, a JSON string containing the extracted and formatted specifications, is then printed.

Context Engineering and Prompt Engineering

Context Engineering (see Fig.1) is the systematic discipline of designing, constructing, and delivering a complete informational environment to an AI model prior to token generation. This methodology asserts that the quality of a model's output is less dependent on the model's architecture itself and more on the richness of the context provided.

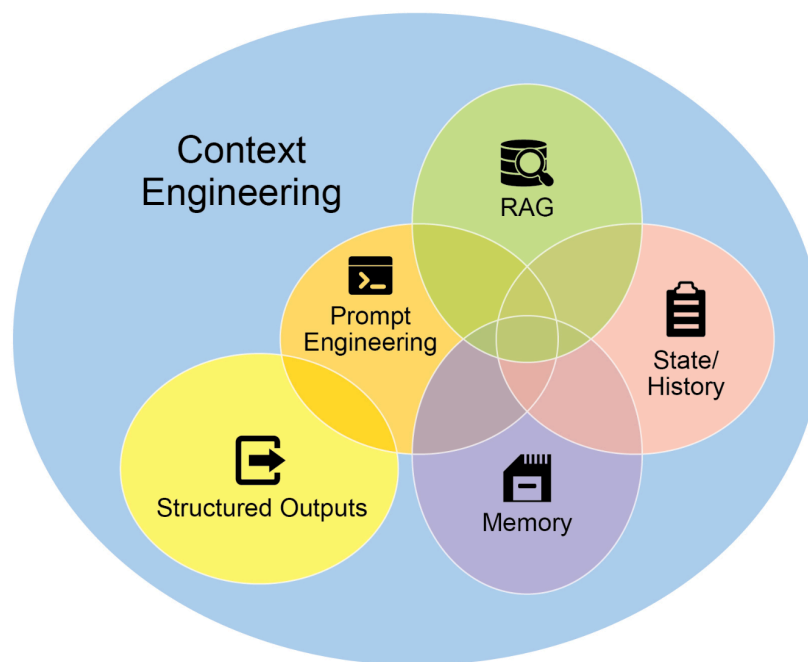


Fig.1: Context Engineering is the discipline of building a rich, comprehensive informational environment for an AI, as the quality of this context is a primary factor in enabling advanced Agentic performance.

It represents a significant evolution from traditional prompt engineering, which focuses primarily on optimizing the phrasing of a user's immediate query. Context Engineering expands this scope to include several layers of information, such as the **system prompt**, which is a foundational set of instructions defining the AI's operational

parameters—for instance, *"You are a technical writer; your tone must be formal and precise."* The context is further enriched with external data. This includes retrieved documents, where the AI actively fetches information from a knowledge base to inform its response, such as pulling technical specifications for a project. It also incorporates tool outputs, which are the results from the AI using an external API to obtain real-time data, like querying a calendar to determine a user's availability. This explicit data is combined with critical implicit data, such as user identity, interaction history, and environmental state. The core principle is that even advanced models underperform when provided with a limited or poorly constructed view of the operational environment.

This practice, therefore, reframes the task from merely answering a question to building a comprehensive operational picture for the agent. For example, a context-engineered agent would not just respond to a query but would first integrate the user's calendar availability (a tool output), the professional relationship with an email's recipient (implicit data), and notes from previous meetings (retrieved documents). This allows the model to generate outputs that are highly relevant, personalized, and pragmatically useful. The "engineering" component involves creating robust pipelines to fetch and transform this data at runtime and establishing feedback loops to continually improve context quality.

To implement this, specialized tuning systems can be used to automate the improvement process at scale. For example, tools like Google's Vertex AI prompt optimizer can enhance model performance by systematically evaluating responses against a set of sample inputs and predefined evaluation metrics. This approach is effective for adapting prompts and system instructions across different models without requiring extensive manual rewriting. By providing such an optimizer with sample prompts, system instructions, and a template, it can programmatically refine the contextual inputs, offering a structured method for implementing the feedback loops required for sophisticated Context Engineering.

This structured approach is what differentiates a rudimentary AI tool from a more sophisticated and contextually-aware system. It treats the context itself as a primary component, placing critical importance on what the agent knows, when it knows it, and how it uses that information. The practice ensures the model has a well-rounded understanding of the user's intent, history, and current environment. Ultimately, Context Engineering is a crucial methodology for advancing stateless chatbots into highly capable, situationally-aware systems.

At a Glance

What: Complex tasks often overwhelm LLMs when handled within a single prompt, leading to significant performance issues. The cognitive load on the model increases

the likelihood of errors such as overlooking instructions, losing context, and generating incorrect information. A monolithic prompt struggles to manage multiple constraints and sequential reasoning steps effectively. This results in unreliable and inaccurate outputs, as the LLM fails to address all facets of the multifaceted request.

Why: Prompt chaining provides a standardized solution by breaking down a complex problem into a sequence of smaller, interconnected sub-tasks. Each step in the chain uses a focused prompt to perform a specific operation, significantly improving reliability and control. The output from one prompt is passed as the input to the next, creating a logical workflow that progressively builds towards the final solution. This modular, divide-and-conquer strategy makes the process more manageable, easier to debug, and allows for the integration of external tools or structured data formats between steps. This pattern is foundational for developing sophisticated, multi-step Agentic systems that can plan, reason, and execute complex workflows.

Rule of thumb: Use this pattern when a task is too complex for a single prompt, involves multiple distinct processing stages, requires interaction with external tools between steps, or when building Agentic systems that need to perform multi-step reasoning and maintain state.

Visual summary

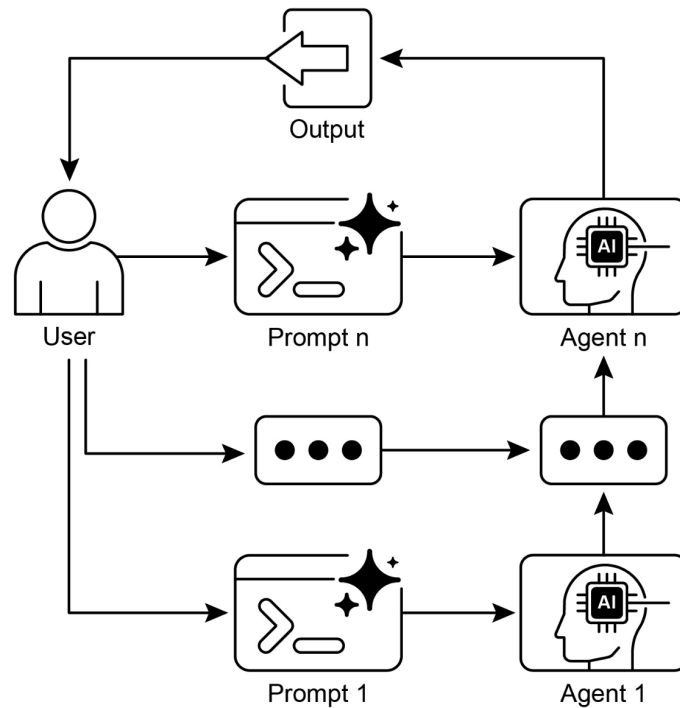


Fig. 2: Prompt Chaining Pattern: Agents receive a series of prompts from the user, with the output of each agent serving as the input for the next in the chain.

Key Takeaways

Here are some key takeaways:

- Prompt Chaining breaks down complex tasks into a sequence of smaller, focused steps. This is occasionally known as the Pipeline pattern.
- Each step in a chain involves an LLM call or processing logic, using the output of the previous step as input.
- This pattern improves the reliability and manageability of complex interactions with language models.
- Frameworks like LangChain/LangGraph, and Google ADK provide robust tools to define, manage, and execute these multi-step sequences.

Conclusion

By deconstructing complex problems into a sequence of simpler, more manageable sub-tasks, prompt chaining provides a robust framework for guiding large language models. This "divide-and-conquer" strategy significantly enhances the reliability and control of the output by focusing the model on one specific operation at a time. As a foundational pattern, it enables the development of sophisticated AI agents capable of multi-step reasoning, tool integration, and state management. Ultimately, mastering prompt chaining is crucial for building robust, context-aware systems that can execute intricate workflows well beyond the capabilities of a single prompt.

References

1. LangChain Documentation on LCEL: https://python.langchain.com/v0.2/docs/core_modules/expression_language/
2. LangGraph Documentation: <https://langchain-ai.github.io/langgraph/>
3. Prompt Engineering Guide - Chaining Prompts: <https://www.promptingguide.ai/techniques/chaining>
4. OpenAI API Documentation (General Prompting Concepts): <https://platform.openai.com/docs/guides/gpt/prompting>
5. Crew AI Documentation (Tasks and Processes): <https://docs.crewai.com/>
6. Google AI for Developers (Prompting Guides): <https://cloud.google.com/discover/what-is-prompt-engineering?hl=en>
7. Vertex Prompt Optimizer <https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/prompt-optimizer>

Chapter 2: Routing

Routing Pattern Overview

While sequential processing via prompt chaining is a foundational technique for executing deterministic, linear workflows with language models, its applicability is limited in scenarios requiring adaptive responses. Real-world agentic systems must often arbitrate between multiple potential actions based on contingent factors, such as the state of the environment, user input, or the outcome of a preceding operation. This capacity for dynamic decision-making, which governs the flow of control to different specialized functions, tools, or sub-processes, is achieved through a mechanism known as routing.

Routing introduces conditional logic into an agent's operational framework, enabling a shift from a fixed execution path to a model where the agent dynamically evaluates specific criteria to select from a set of possible subsequent actions. This allows for more flexible and context-aware system behavior.

For instance, an agent designed for customer inquiries, when equipped with a routing function, can first classify an incoming query to determine the user's intent. Based on this classification, it can then direct the query to a specialized agent for direct question-answering, a database retrieval tool for account information, or an escalation procedure for complex issues, rather than defaulting to a single, predetermined response pathway. Therefore, a more sophisticated agent using routing could:

1. Analyze the user's query.
2. **Route** the query based on its *intent*:
 - If the intent is "check order status", route to a sub-agent or tool chain that interacts with the order database.
 - If the intent is "product information", route to a sub-agent or chain that searches the product catalog.
 - If the intent is "technical support", route to a different chain that accesses troubleshooting guides or escalates to a human.
 - If the intent is unclear, route to a clarification sub-agent or prompt chain.

The core component of the Routing pattern is a mechanism that performs the evaluation and directs the flow. This mechanism can be implemented in several ways:

- **LLM-based Routing:** The language model itself can be prompted to analyze the input and output a specific identifier or instruction that indicates the next step or destination. For example, a prompt might ask the LLM to "Analyze the following

user query and output only the category: 'Order Status', 'Product Info', 'Technical Support', or 'Other'." The agentic system then reads this output and directs the workflow accordingly.

- **Embedding-based Routing:** The input query can be converted into a vector embedding (see RAG, Chapter 14). This embedding is then compared to embeddings representing different routes or capabilities. The query is routed to the route whose embedding is most similar. This is useful for semantic routing, where the decision is based on the meaning of the input rather than just keywords.
- **Rule-based Routing:** This involves using predefined rules or logic (e.g., if-else statements, switch cases) based on keywords, patterns, or structured data extracted from the input. This can be faster and more deterministic than LLM-based routing, but is less flexible for handling nuanced or novel inputs.
- **Machine Learning Model-Based Routing:** it employs a discriminative model, such as a classifier, that has been specifically trained on a small corpus of labeled data to perform a routing task. While it shares conceptual similarities with embedding-based methods, its key characteristic is the supervised fine-tuning process, which adjusts the model's parameters to create a specialized routing function. This technique is distinct from LLM-based routing because the decision-making component is not a generative model executing a prompt at inference time. Instead, the routing logic is encoded within the fine-tuned model's learned weights. While LLMs may be used in a pre-processing step to generate synthetic data for augmenting the training set, they are not involved in the real-time routing decision itself.

Routing mechanisms can be implemented at multiple junctures within an agent's operational cycle. They can be applied at the outset to classify a primary task, at intermediate points within a processing chain to determine a subsequent action, or during a subroutine to select the most appropriate tool from a given set.

Computational frameworks such as LangChain, LangGraph, and Google's Agent Developer Kit (ADK) provide explicit constructs for defining and managing such conditional logic. With its state-based graph architecture, LangGraph is particularly well-suited for complex routing scenarios where decisions are contingent upon the accumulated state of the entire system. Similarly, Google's ADK provides foundational components for structuring an agent's capabilities and interaction models, which serve as the basis for implementing routing logic. Within the execution environments provided by these frameworks, developers define the possible operational paths and

the functions or model-based evaluations that dictate the transitions between nodes in the computational graph.

The implementation of routing enables a system to move beyond deterministic sequential processing. It facilitates the development of more adaptive execution flows that can respond dynamically and appropriately to a wider range of inputs and state changes.

Practical Applications & Use Cases

The routing pattern is a critical control mechanism in the design of adaptive agentic systems, enabling them to dynamically alter their execution path in response to variable inputs and internal states. Its utility spans multiple domains by providing a necessary layer of conditional logic.

In human-computer interaction, such as with virtual assistants or AI-driven tutors, routing is employed to interpret user intent. An initial analysis of a natural language query determines the most appropriate subsequent action, whether it is invoking a specific information retrieval tool, escalating to a human operator, or selecting the next module in a curriculum based on user performance. This allows the system to move beyond linear dialogue flows and respond contextually.

Within automated data and document processing pipelines, routing serves as a classification and distribution function. Incoming data, such as emails, support tickets, or API payloads, is analyzed based on content, metadata, or format. The system then directs each item to a corresponding workflow, such as a sales lead ingestion process, a specific data transformation function for JSON or CSV formats, or an urgent issue escalation path.

In complex systems involving multiple specialized tools or agents, routing acts as a high-level dispatcher. A research system composed of distinct agents for searching, summarizing, and analyzing information would use a router to assign tasks to the most suitable agent based on the current objective. Similarly, an AI coding assistant uses routing to identify the programming language and user's intent—to debug, explain, or translate—before passing a code snippet to the correct specialized tool.

Ultimately, routing provides the capacity for logical arbitration that is essential for creating functionally diverse and context-aware systems. It transforms an agent from a static executor of pre-defined sequences into a dynamic system that can make

decisions about the most effective method for accomplishing a task under changing conditions.

Hands-On Code Example (LangChain)

Implementing routing in code involves defining the possible paths and the logic that decides which path to take. Frameworks like LangChain and LangGraph provide specific components and structures for this. LangGraph's state-based graph structure is particularly intuitive for visualizing and implementing routing logic.

This code demonstrates a simple agent-like system using LangChain and Google's Generative AI. It sets up a "coordinator" that routes user requests to different simulated "sub-agent" handlers based on the request's intent (booking, information, or unclear). The system uses a language model to classify the request and then delegates it to the appropriate handler function, simulating a basic delegation pattern often seen in multi-agent architectures.

First, ensure you have the necessary libraries installed:

```
pip install langchain langgraph google-cloud-aiplatform
langchain-google-genai google-adk deprecated pydantic
```

You will also need to set up your environment with your API key for the language model you choose (e.g., OpenAI, Google Gemini, Anthropic).

```
# Copyright (c) 2025 Marco Fago
# https://www.linkedin.com/in/marco-fago/
#
# This code is licensed under the MIT License.
# See the LICENSE file in the repository for the full license text.

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough,
RunnableBranch

# --- Configuration ---
# Ensure your API key environment variable is set (e.g.,
GOOGLE_API_KEY)
try:
    llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash",
```



```

temperature=0)
    print(f"Language model initialized: {llm.model}")
except Exception as e:
    print(f"Error initializing language model: {e}")
    llm = None

# --- Define Simulated Sub-Agent Handlers (equivalent to ADK
sub_agents) ---

def booking_handler(request: str) -> str:
    """Simulates the Booking Agent handling a request."""
    print("\n--- DELEGATING TO BOOKING HANDLER ---")
    return f"Booking Handler processed request: '{request}'. Result:
Simulated booking action."

def info_handler(request: str) -> str:
    """Simulates the Info Agent handling a request."""
    print("\n--- DELEGATING TO INFO HANDLER ---")
    return f"Info Handler processed request: '{request}'. Result:
Simulated information retrieval."

def unclear_handler(request: str) -> str:
    """Handles requests that couldn't be delegated."""
    print("\n--- HANDLING UNCLEAR REQUEST ---")
    return f"Coordinator could not delegate request: '{request}'.
Please clarify."

# --- Define Coordinator Router Chain (equivalent to ADK
coordinator's instruction) ---
# This chain decides which handler to delegate to.
coordinator_router_prompt = ChatPromptTemplate.from_messages([
    ("system", """Analyze the user's request and determine which
specialist handler should process it.
    - If the request is related to booking flights or hotels,
    output 'booker'.
    - For all other general information questions, output 'info'.
    - If the request is unclear or doesn't fit either category,
    output 'unclear'.
    ONLY output one word: 'booker', 'info', or 'unclear'."""),
    ("user", "{request}")
])

if llm:
    coordinator_router_chain = coordinator_router_prompt | llm |
StrOutputParser()

# --- Define the Delegation Logic (equivalent to ADK's Auto-Flow

```

```

based on sub_agents) ---
# Use RunnableBranch to route based on the router chain's output.

# Define the branches for the RunnableBranch
branches = {
    "booker": RunnablePassthrough.assign(output=lambda x:
booking_handler(x['request']['request'])),
    "info": RunnablePassthrough.assign(output=lambda x:
info_handler(x['request']['request'])),
    "unclear": RunnablePassthrough.assign(output=lambda x:
unclear_handler(x['request']['request'])),
}

# Create the RunnableBranch. It takes the output of the router chain
# and routes the original input ('request') to the corresponding
handler.
delegation_branch = RunnableBranch(
    (lambda x: x['decision'].strip() == 'booker', branches["booker"]),
# Added .strip()
    (lambda x: x['decision'].strip() == 'info', branches["info"]),
# Added .strip()
    branches["unclear"] # Default branch for 'unclear' or any other
output
)

# Combine the router chain and the delegation branch into a single
runnable
# The router chain's output ('decision') is passed along with the
original input ('request')
# to the delegation_branch.
coordinator_agent = {
    "decision": coordinator_router_chain,
    "request": RunnablePassthrough()
} | delegation_branch | (lambda x: x['output']) # Extract the final
output

# --- Example Usage ---
def main():
    if not llm:
        print("\nSkipping execution due to LLM initialization
failure.")
        return

    print("--- Running with a booking request ---")
    request_a = "Book me a flight to London."
    result_a = coordinator_agent.invoke({"request": request_a})
    print(f"Final Result A: {result_a}")

```

```

print("\n--- Running with an info request ---")
request_b = "What is the capital of Italy?"
result_b = coordinator_agent.invoke({"request": request_b})
print(f"Final Result B: {result_b}")

print("\n--- Running with an unclear request ---")
request_c = "Tell me about quantum physics."
result_c = coordinator_agent.invoke({"request": request_c})
print(f"Final Result C: {result_c}")

if __name__ == "__main__":
    main()

```

As mentioned, this Python code constructs a simple agent-like system using the LangChain library and Google's Generative AI model, specifically gemini-2.5-flash. In detail, It defines three simulated sub-agent handlers: `booking_handler`, `info_handler`, and `unclear_handler`, each designed to process specific types of requests.

A core component is the `coordinator_router_chain`, which utilizes a `ChatPromptTemplate` to instruct the language model to categorize incoming user requests into one of three categories: 'booker', 'info', or 'unclear'. The output of this router chain is then used by a `RunnableBranch` to delegate the original request to the corresponding handler function. The `RunnableBranch` checks the decision from the language model and directs the request data to either the `booking_handler`, `info_handler`, or `unclear_handler`. The `coordinator_agent` combines these components, first routing the request for a decision and then passing the request to the chosen handler. The final output is extracted from the handler's response.

The main function demonstrates the system's usage with three example requests, showcasing how different inputs are routed and processed by the simulated agents. Error handling for language model initialization is included to ensure robustness. The code structure mimics a basic multi-agent framework where a central coordinator delegates tasks to specialized agents based on intent.

Hands-On Code Example (Google ADK)

The Agent Development Kit (ADK) is a framework for engineering agentic systems, providing a structured environment for defining an agent's capabilities and behaviours. In contrast to architectures based on explicit computational graphs,

routing within the ADK paradigm is typically implemented by defining a discrete set of "tools" that represent the agent's functions. The selection of the appropriate tool in response to a user query is managed by the framework's internal logic, which leverages an underlying model to match user intent to the correct functional handler.

This Python code demonstrates an example of an Agent Development Kit (ADK) application using Google's ADK library. It sets up a "Coordinator" agent that routes user requests to specialized sub-agents ("Booker" for bookings and "Info" for general information) based on defined instructions. The sub-agents then use specific tools to simulate handling the requests, showcasing a basic delegation pattern within an agent system

```
# Copyright (c) 2025 Marco Fago
#
# This code is licensed under the MIT License.
# See the LICENSE file in the repository for the full license text.

import uuid
from typing import Dict, Any, Optional

from google.adk.agents import Agent
from google.adk.runners import InMemoryRunner
from google.adk.tools import FunctionTool
from google.genai import types
from google.adk.events import Event

# --- Define Tool Functions ---
# These functions simulate the actions of the specialist agents.

def booking_handler(request: str) -> str:
    """
    Handles booking requests for flights and hotels.
    Args:
        request: The user's request for a booking.
    Returns:
        A confirmation message that the booking was handled.
    """
    print("----- Booking Handler Called -----")
    return f"Booking action for '{request}' has been simulated."

def info_handler(request: str) -> str:
    """
    Handles general information requests.
    Args:
```

```

        request: The user's question.
Returns:
    A message indicating the information request was handled.
    """
    print("----- Info Handler Called
-----")
    return f"Information request for '{request}'. Result: Simulated
information retrieval."

def unclear_handler(request: str) -> str:
    """Handles requests that couldn't be delegated."""
    return f"Coordinator could not delegate request: '{request}'.
Please clarify."

# --- Create Tools from Functions ---
booking_tool = FunctionTool(booking_handler)
info_tool = FunctionTool(info_handler)

# Define specialized sub-agents equipped with their respective tools
booking_agent = Agent(
    name="Booker",
    model="gemini-2.0-flash",
    description="A specialized agent that handles all flight
        and hotel booking requests by calling the booking tool.",
    tools=[booking_tool]
)

info_agent = Agent(
    name="Info",
    model="gemini-2.0-flash",
    description="A specialized agent that provides general information
        and answers user questions by calling the info tool.",
    tools=[info_tool]
)

# Define the parent agent with explicit delegation instructions
coordinator = Agent(
    name="Coordinator",
    model="gemini-2.0-flash",
    instruction=(
        "You are the main coordinator. Your only task is to analyze
        incoming user requests "
        "and delegate them to the appropriate specialist agent.
        Do not try to answer the user directly.\n"
        "- For any requests related to booking flights or hotels,
        delegate to the 'Booker' agent.\n"
        "- For all other general information questions, delegate to

```

```

the 'Info' agent."
    ),
    description="A coordinator that routes user requests to the
        correct specialist agent.",
    # The presence of sub_agents enables LLM-driven delegation
    (Auto-Flow) by default.
    sub_agents=[booking_agent, info_agent]
)

# --- Execution Logic ---

async
def run_coordinator(runner: InMemoryRunner, request: str):
    """Runs the coordinator agent with a given request and
    delegates."""
    print(f"\n--- Running Coordinator with request: '{request}' ---")
    final_result = ""
    try:
        user_id = "user_123"
        session_id = str(uuid.uuid4())
        await
        runner.session_service.create_session(
            app_name=runner.app_name, user_id=user_id,
            session_id=session_id
        )

        for event in runner.run(
            user_id=user_id,
            session_id=session_id,
            new_message=types.Content(
                role='user',
                parts=[types.Part(text=request)]
            ),
        ):
            if event.is_final_response() and event.content:
                # Try to get text directly from event.content
                # to avoid iterating parts
                if hasattr(event.content, 'text') and
event.content.text:
                    final_result = event.content.text
                elif event.content.parts:
                    # Fallback: Iterate through parts and extract text
                    (might trigger warning)
                    text_parts = [part.text for part in
event.content.parts if part.text]
                    final_result = "".join(text_parts)
                # Assuming the loop should break after the final

```

```

response
    break

    print(f"Coordinator Final Response: {final_result}")
    return final_result
except Exception as e:
    print(f"An error occurred while processing your request: {e}")
    return f"An error occurred while processing your request: {e}"

async
def main():
    """Main function to run the ADK example."""
    print("--- Google ADK Routing Example (ADK Auto-Flow Style) ---")
    print("Note: This requires Google ADK installed and
authenticated.")

    runner = InMemoryRunner(coordinator)
    # Example Usage
    result_a = await run_coordinator(runner, "Book me a hotel in
Paris.")
    print(f"Final Output A: {result_a}")
    result_b = await run_coordinator(runner, "What is the highest
mountain in the world?")
    print(f"Final Output B: {result_b}")
    result_c = await run_coordinator(runner, "Tell me a random fact.")
# Should go to Info
    print(f"Final Output C: {result_c}")
    result_d = await run_coordinator(runner, "Find flights to Tokyo
next month.") # Should go to Booker
    print(f"Final Output D: {result_d}")

if __name__ == "__main__":
    import nest_asyncio
    nest_asyncio.apply()
    await main()

```

This script consists of a main Coordinator agent and two specialized sub_agents: Booker and Info. Each specialized agent is equipped with a FunctionTool that wraps a Python function simulating an action. The booking_handler function simulates handling flight and hotel bookings, while the info_handler function simulates retrieving general information. The unclear_handler is included as a fallback for requests the coordinator cannot delegate, although the current coordinator logic doesn't explicitly use it for delegation failure in the main run_coordinator function.

The Coordinator agent's primary role, as defined in its instruction, is to analyze incoming user messages and delegate them to either the Booker or Info agent. This delegation is handled automatically by the ADK's Auto-Flow mechanism because the Coordinator has sub_agents defined. The run_coordinator function sets up an InMemoryRunner, creates a user and session ID, and then uses the runner to process the user's request through the coordinator agent. The runner.run method processes the request and yields events, and the code extracts the final response text from the event.content.

The main function demonstrates the system's usage by running the coordinator with different requests, showcasing how it delegates booking requests to the Booker and information requests to the Info agent.

At a Glance

What: Agentic systems must often respond to a wide variety of inputs and situations that cannot be handled by a single, linear process. A simple sequential workflow lacks the ability to make decisions based on context. Without a mechanism to choose the correct tool or sub-process for a specific task, the system remains rigid and non-adaptive. This limitation makes it difficult to build sophisticated applications that can manage the complexity and variability of real-world user requests.

Why: The Routing pattern provides a standardized solution by introducing conditional logic into an agent's operational framework. It enables the system to first analyze an incoming query to determine its intent or nature. Based on this analysis, the agent dynamically directs the flow of control to the most appropriate specialized tool, function, or sub-agent. This decision can be driven by various methods, including prompting LLMs, applying predefined rules, or using embedding-based semantic similarity. Ultimately, routing transforms a static, predetermined execution path into a flexible and context-aware workflow capable of selecting the best possible action.

Rule of Thumb: Use the Routing pattern when an agent must decide between multiple distinct workflows, tools, or sub-agents based on the user's input or the current state. It is essential for applications that need to triage or classify incoming requests to handle different types of tasks, such as a customer support bot distinguishing between sales inquiries, technical support, and account management questions.

Visual Summary:

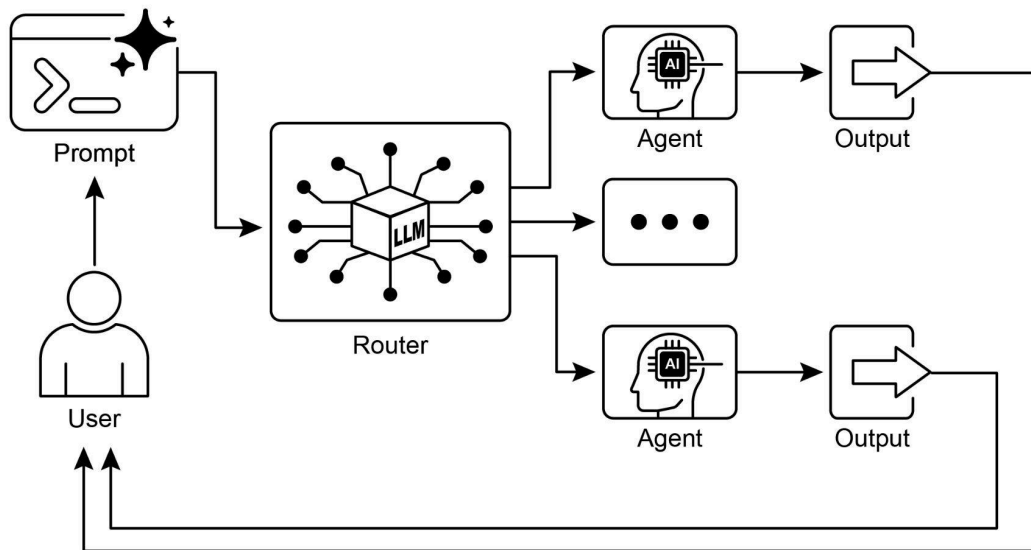


Fig.1: Router pattern, using an LLM as a Router

Key Takeaways

- Routing enables agents to make dynamic decisions about the next step in a workflow based on conditions.
- It allows agents to handle diverse inputs and adapt their behavior, moving beyond linear execution.
- Routing logic can be implemented using LLMs, rule-based systems, or embedding similarity.
- Frameworks like LangGraph and Google ADK provide structured ways to define and manage routing within agent workflows, albeit with different architectural approaches.

Conclusion

The Routing pattern is a critical step in building truly dynamic and responsive agentic systems. By implementing routing, we move beyond simple, linear execution flows and

empower our agents to make intelligent decisions about how to process information, respond to user input, and utilize available tools or sub-agents.

We've seen how routing can be applied in various domains, from customer service chatbots to complex data processing pipelines. The ability to analyze input and conditionally direct the workflow is fundamental to creating agents that can handle the inherent variability of real-world tasks.

The code examples using LangChain and Google ADK demonstrate two different, yet effective, approaches to implementing routing. LangGraph's graph-based structure provides a visual and explicit way to define states and transitions, making it ideal for complex, multi-step workflows with intricate routing logic. Google ADK, on the other hand, often focuses on defining distinct capabilities (Tools) and relies on the framework's ability to route user requests to the appropriate tool handler, which can be simpler for agents with a well-defined set of discrete actions.

Mastering the Routing pattern is essential for building agents that can intelligently navigate different scenarios and provide tailored responses or actions based on context. It's a key component in creating versatile and robust agentic applications.

References

1. LangGraph Documentation: <https://www.langchain.com/>
2. Google Agent Developer Kit Documentation: <https://google.github.io/adk-docs/>

Chapter 3: Parallelization

Parallelization Pattern Overview

In the previous chapters, we've explored Prompt Chaining for sequential workflows and Routing for dynamic decision-making and transitions between different paths. While these patterns are essential, many complex agentic tasks involve multiple sub-tasks that can be executed *simultaneously* rather than one after another. This is where the **Parallelization** pattern becomes crucial.

Parallelization involves executing multiple components, such as LLM calls, tool usages, or even entire sub-agents, concurrently (see Fig.1). Instead of waiting for one step to complete before starting the next, parallel execution allows independent tasks to run at the same time, significantly reducing the overall execution time for tasks that can be broken down into independent parts.

Consider an agent designed to research a topic and summarize its findings. A sequential approach might:

1. Search for Source A.
2. Summarize Source A.
3. Search for Source B.
4. Summarize Source B.
5. Synthesize a final answer from summaries A and B.

A parallel approach could instead:

1. Search for Source A *and* Search for Source B simultaneously.
2. Once both searches are complete, Summarize Source A *and* Summarize Source B simultaneously.
3. Synthesize a final answer from summaries A and B (this step is typically sequential, waiting for the parallel steps to finish).

The core idea is to identify parts of the workflow that do not depend on the output of other parts and execute them in parallel. This is particularly effective when dealing with external services (like APIs or databases) that have latency, as you can issue multiple requests concurrently.

Implementing parallelization often requires frameworks that support asynchronous execution or multi-threading/multi-processing. Modern agentic frameworks are